OrchLang: Replayable Verification of Declared LLM Usage Observations

Author names pending*

Affiliations and postal addresses pending

Abstract

Equal resource budgets do not establish confidentiality of resource observations. OrchLang is research software for request analysis and conditional usage-law verification in language-model workflows. It combines a bounded compiler, request graphs, rational outcome models and independent certificate replay. Shared-prefix construction and replay resolve a resource limit at 128 calls per secret arm in a controlled family; diagnostics preserve unknown outcomes. A machine-checked theorem covers request-history and local-result agreement for a normalized straight-line fragment under an atomic-read premise. A frozen external-source study retains 59 original candidate files across 12 repositories, with none admitted by the narrow Python frontend. The contribution is an inspectable specialization of established relational and automata methods, with measured software evidence and explicit source, observer and provider boundaries.

Keywords: research software, relational verification, LLM workflows, usage observations, reproducibility

1. Motivation and significance

An observer who knows the public context can learn from an execution's call count, input usage, output usage or stop position. Equal worst-case caps do not ensure equal distributions of these observations. Conversely, different requests can have the same usage law in a specified model. OrchLang lets researchers distinguish these questions and inspect the assumptions supporting a result.

Resource-aware information flow and relational cost analysis are established techniques [1, 2]. Probabilistic-automata equivalence also predates this

*Correspondence pending. Author placeholders retained by request; this research draft has not been submitted.

software [3]. PPDL models uncertain multi-call LLM flows [4]; Agentproof extracts workflow graphs and checks structural and temporal policies [5]. Recent agent IFC systems address disclosure through tool interfaces [6, 7], while ProMCP profiles token and latency costs [8]. OrchLang specializes request identity and a declared usage observer, rather than claiming a new general algorithm or complete agent confidentiality. Prompt-cache timing is a concrete observer outside its model [9].

2. Software description

2.1. Architecture and functionality

The C++17 compiler checks typed inputs, resolved model contracts, flow policies, literal-bounded retries and conditional token bounds. A bridge exports its admitted request/control fragment. A separate Python frontend accepts a documented static LangGraph subset; unsupported statements, effects, reducers and parallel edges reject. Direct graph JSON supports productive cycles as finite-prefix properties. Assignment invalidates stale result relations, and request comparison retains exact templates, ordered typed arguments and corresponding prior results.

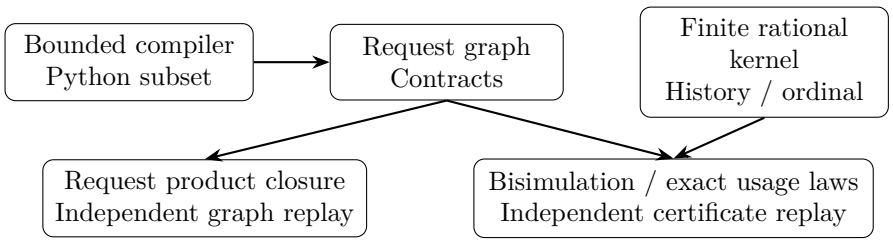


Figure 1: Source adapters and model premises remain distinct from independently replayed graph and usage-law evidence.

The quantitative tools use successful outcomes from an explicit finite rational kernel. For one public store and the supplied secret assignments, call labels contain the normalized contract and integer input/output usage. Stopped and still-callable mass are separate. Strong bisimulation is sufficient; exact trace-law analysis can decide cases it leaves inconclusive. The latter explores the rational span of μA_w, where μ is the initial scenario difference and A_w composes observable-label matrices [3]. Replay separately reconstructs the semantic system and checks basis words, independence, closure and output columns, or a finite distinguishing witness.

2.2. Source and resource boundaries

Version 0.5.0rc2 rejects node names reserved by LangGraph 1.2.14 [10] and module functions shadowing `str`/`bool` annotations, including Python 3.14's deferred lookup [11]. Its normalized-IR theorem compares node-entry snapshot reads with eager lowered registers: if later requests never read a field already written by a returned-result call, histories and local results agree for every deterministic history/request oracle. Both theorem checks are closed under the global context. AST extraction, final returned states and whole-graph refinement are not mechanized.

Version-2 trace reports attach bounded reason/phase diagnostics and work counters to unknown outcomes, without a partial system or proof. Replay accepts historical version-1 reports. Certificate prefixes are independently reconstructed once in a trie, preserving exact row checks. Adjacency and parsed-row storage are reserved before allocation. The meters estimate deterministic work; they are not RSS guarantees.

3. Illustrative examples

The authored `different_requests_equal_usage` fixture chooses template `A` or `B` by a secret bit. Each has modeled input usage one; a declared kernel gives output usage one or two with probability 1/2 each. Request equality rejects, but usage-law equality proves and replays. This validates a modeled observer choice, not a provider law.

Authored case	Request	Bisim.	Exact law
Different requests, equal usage	Reject	Equal	Equal
Equal means, distinct laws	Reject	Inconcl.	Unequal
Aggregate hides order	Reject	Inconcl.	Unequal

Table 1: The same fixed graph/kernel inputs are used across checkers. Rejection and inconclusiveness concern sufficient criteria, not proved leakage.

For equal means, one output law is $\{1, 3\}$ with equal probabilities and the other is always two: total variation is one at horizon one. The order fixture emits $(1, 3)$ or $(3, 1)$; aggregate total variation is zero, ordered-trace total variation one at horizon two. Across all twelve authored fixtures, request checking accepts three, bisimulation proves five equal, and exact analysis yields six equal/six unequal; all twelve exact results replay.

4. Evaluation and reproducibility

The frozen run includes 44 core stages, 112 native cases in each optimized and sanitizer run, 230 Python audit tests, 23 isolated package stages, and 11/22 pinned LangGraph backend/frontend tests. The separate unchanged-source incident extension passes sixteen stages. These are software checks, not proofs of complete implementation correctness or independent human artifact evaluation.

4.1. Controlled scaling

Seven symmetric chain sizes (2–256 calls per secret arm) use identical frozen inputs, one warmup and five measured repetitions in separate processes per size. The solver baseline is revision `5aa4ebc`; current inputs are identical. Both versions complete through 64 calls. At 128 the old producer reaches its replay-preflight cap after closure; prefix sharing now produces and independently verifies equality. Both remain unknown at 256, with a diagnosed residual work limit.

At 64, replay median is 138.51 ms before and 87.51 ms after (36.8% lower in this run). Reported replay meters are 9,972,749 and 1,784,025 bytes. The new meter additionally charges retained parsed-row storage; these values are not measured memory. Current 128-call medians are 82.79 ms for analysis and 318.68 ms for replay; no old replay-time comparison exists because no certificate was emitted. Timers exclude imports, RSS includes runtime startup, and raw min/max samples, certificate sizes and memory plots are archived. One synthetic family establishes neither broad scalability nor lower memory use.

4.2. External source and measurement

A protocol frozen before admission outcomes selected twelve repositories using GitHub search and reported SPDX identifiers. A post-freeze audit against SPDX 3.29.0 OSI flags confirmed the manual allowlist and ordered cohort. A repository-root path-enumeration clarification was logged after outcomes; sources and denominators were unchanged. Enumeration retained 4,348 Python paths, 59 AST-selected candidate files from nine repositories, and one separately recorded non-candidate parse error. All 59 originals are unsupported by the evaluated grammar; no external-source differential ran. First rejections comprise 34 unsupported imports, 19 top-level import statements, four annotation forms and two top-level conditionals. This convenience sample includes tutorials and infrastructure-like files; it provides no prevalence estimate or broad framework-coverage claim.

A separate pinned incident-triage integration executes 22 unchanged-source offline cases and 45 mock calls. Its controlled shared-meter witness has 456 unique estimated usage units but 687 when returned snapshots are summed. A bounded isolation wrapper and replay reproduce that source-specific finding; they do not prove general concurrency safety. Local `tiktoken` 0.12.0 [12] calibration on 90 paired call texts and twelve authored controls distinguishes character estimates from three BPE counts. No provider envelopes, cache discounts, usage invoices or stochastic provider premises are validated.

	Code metadata description	Metadata
C1	Current code version	0.5.0rc2 research candidate
C2	Permanent link to code/repository used for this code version	`dd97af2ac714f8c0dbf5ddb63ac29cbdb4a2d0dc`
C3	Permanent link to Reproducible Capsule	Hash-bound source/evidence archives prepared; persistent capsule identifier pending
C4	Legal Code License	MIT; upstream snapshot licenses retained
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	C++17; Python 3.10+ core; optional LangGraph; Rocq 9.3.0 proof check
C7	Compilation requirements, operating environments and dependencies	macOS recorded run; make/C++17 and Python; optional locked LangGraph, tokenizer and plotting environments
C8	If available, link to developer documentation/manual	Publication reproduction guide and developer documentation
C9	Support email for questions	Author email pending; public issue tracker

Table 2: Software metadata. Code identity is distinct from the later paper/evidence commit. Author fields remain explicit placeholders.

5. Impact and limitations

The software provides eight installed analysis and replay commands, exact witnesses and explicit unknown outcomes for workflow usage research. External adoption is unmeasured. Current source adapters are narrow. Broader source support, refinement of complete programs, error and retry models, provider accounting and independent human reproduction remain necessary extensions. Usage-law equality excludes output content, timing,

transport and persistent-store privacy; it does not establish termination or whole-cycle budgets.

Declarations and software availability

Author identities, affiliations, correspondence, CRediT, funding and competing interests await author completion. Codex and GPT-6 Luna agents assisted research, implementation, tests, proof and drafting. Final human review and responsibility have not been asserted. This draft has not been submitted. The declared software revision includes every named source study; fresh evidence and the companion paper are bound by the publication manifest. Documentation describes one-command reproduction and separate optional environments.

References

[1] V.C. Ngo et al., Verifying and Synthesizing Constant-Resource Implementations with Types, IEEE S&P (2017). doi:10.1109/SP.2017.53.

[2] E. Çiçek et al., Relational Cost Analysis, POPL (2017). doi:10.1145/3009837.3009858.

[3] S. Kiefer et al., Language Equivalence for Probabilistic Automata, CAV (2011), 526–540. doi:10.1007/978-3-642-22110-1_42.

[4] L. Mandel et al., PPDL: LLM-Based Flows as Probabilistic Programs, ICML, PMLR 306 (2026). Proceedings.

[5] M. Xavier et al., Agentproof: Static Verification of Agent Workflow Graphs, arXiv:2603.20356v1 (2026), preprint.

[6] A. Kravchenko et al., APPA: Recoverable Information-Flow Control for Real-World LLM Agents, arXiv:2607.24625v2 (2026), preprint.

[7] M. Shim et al., Confuse the Model, Control the Flow: Understanding and Mitigating Privacy Leakage from LLM Agents with Information Flow Control, arXiv:2609.14003v1 (2026), preprint.

[8] S. Anjum et al., ProMCP: Profiling Token Flows and Latency Costs in Model Context Protocol-Based LLM Agents, Findings of ACL (2026). doi:10.18653/v1/2026.findings-acl.1967.

[9] C. Gu et al., Auditing Prompt Caching in Language Model APIs, ICML, PMLR 267 (2025). Proceedings.

[10] LangChain, LangGraph 1.2.14, tagged implementation. StateGraph source.

[11] L. Hastings, PEP 649: Deferred Evaluation of Annotations Using Descriptors. Python enhancement proposal.

[12] OpenAI, tiktoken 0.12.0, software release. Tagged release.